

Abstracción y Polimorfismo

Abstracción: Proceso mental para extraer características esenciales ignorando detalles irrelevantes según el contexto.

Enfoque en Java: La abstracción permite centrarse en qué hace un objeto en lugar de cómo lo hace.

Definición de Polimorfismo: Capacidad de una variable de referencia para adoptar múltiples formas y responder a mensajes de formas distintas.

Uso de Referencias: Permite que una referencia de superclase se refiera a objetos de diversas subclases de forma genérica.

Profundizando en el Polimorfismo

Sobrecarga (Overloading): Polimorfismo estático resuelto en compilación; ocurre en la misma clase con diferentes firmas de método.

Sobrescritura (Overriding): Redefinición de un método heredado en una subclase manteniendo la misma firma.

Enlace Dinámico: Mecanismo en tiempo de ejecución que decide qué implementación ejecutar basándose en el tipo real del objeto.

Principio de Sustitución: Un objeto de una subclase debe poder reemplazar a uno de su superclase (regla "Es un").

Clases y Métodos Abstractos

Clases Abstractas: Funcionan como esqueletos o plantillas que no pueden ser instanciadas directamente.

Métodos Abstractos: Declaraciones de métodos sin cuerpo que obligan a las subclases concretas a implementarlos.

Consecuencia de Abstracción: Si una clase posee al menos un método abstracto, la clase completa debe declararse como abstracta.

Diseño Flexible: Centraliza el comportamiento común en la superclase y delega lo específico a las subclases.

Interfaces: Contratos

Definición de Interfaz: Tipo abstracto que representa un contrato legal de métodos que una clase se compromete a implementar.

Contenido Tradicional: Las interfaces suelen contener únicamente firmas de métodos y constantes estáticas finales.

Implementación Múltiple: Java permite que una clase implemente varias interfaces, permitiendo que un objeto asuma múltiples "roles".

Diferencia Fundamental: La clase abstracta define lo que un objeto "es", mientras la interfaz define lo que "puede hacer".

Principios de Diseño OO



Alta Cohesión: Grado en que una clase se enfoca en una única tarea clara con métodos relacionados entre sí.

Bajo Acoplamiento: Minimización de dependencias entre clases para que los cambios no afecten a otros módulos.

Responsabilidad Única (SRP): Establece que una clase o función debe tener una sola razón para cambiar.

Reducción de Acoplamiento: El polimorfismo se identifica como la herramienta principal para lograr un bajo acoplamiento.



Patrones y Desacoplamiento

Patrón Strategy: Permite encapsular una familia de algoritmos y hacerlos intercambiables delegando a un objeto a través de una interfaz.

Cambio en Ejecución: El patrón Strategy facilita cambiar el comportamiento de un objeto inyectando diferentes implementaciones.

Desacoplamiento mediante Interfaces: Rompe la dependencia directa entre clases, permitiendo que el sistema sea escalable.

Buenas Prácticas



Nombres e Intención: Las variables y métodos deben indicar claramente su función; se usa PascalCase para clases y camelCase para métodos.

Principio KISS: Se debe buscar siempre la solución más simple y clara, evitando complejidad innecesaria.

Principio DRY: Evita la duplicación de lógica para facilitar el mantenimiento y prevenir errores.

Encapsulamiento y Estado: Los campos deben ser privados y accederse mediante getters/setters para proteger el estado interno.

Inmutabilidad: Preferencia por objetos cuyos campos no cambien tras su construcción para garantizar estados válidos.

